

Classification d'images CIFAR-10 par réseaux CNN

Mohamed-Amine Rguigue - E3FD

Sommaire

- Le problème et le dataset
- Préparation des données
- Modèle 1 – Architecture de base et overfitting
- Modèles 2, 3, 4 – Techniques de régularisation
- Comparaison des modèles
- Analyse détaillée du meilleur modèle
- Limites et pistes d'amélioration
- Conclusion

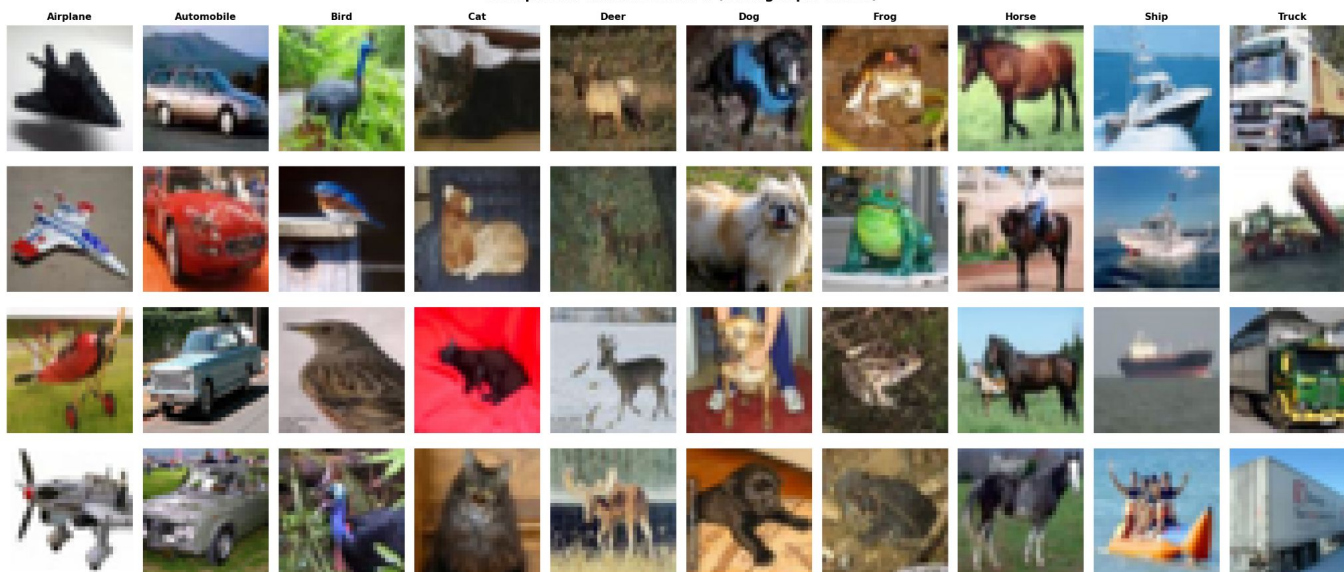


Le problème

Objectif : classer automatiquement une image parmi 10 classes

Pourquoi c'est difficile : images 32×32 pixels, très basse résolution

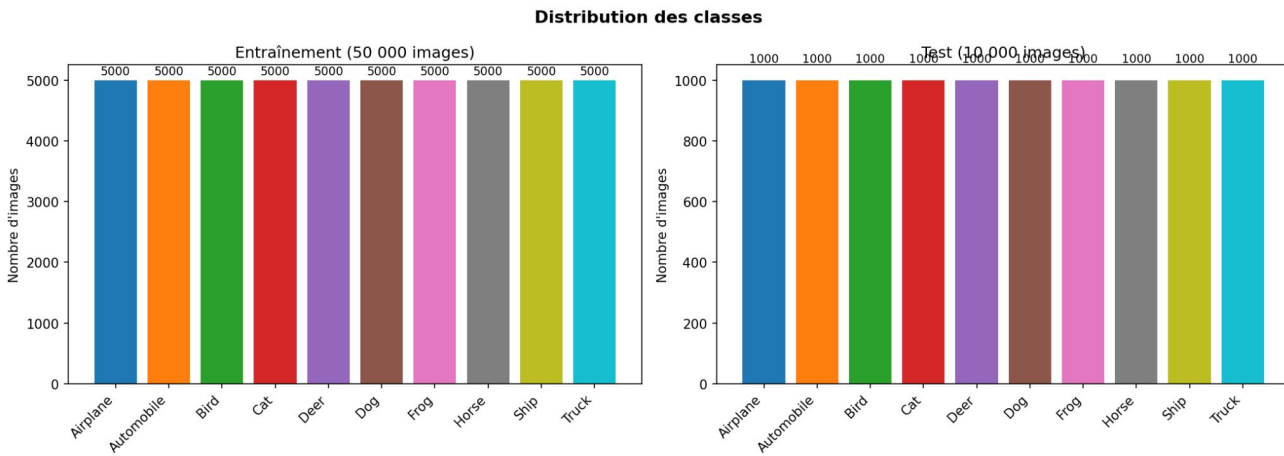
Exemples du dataset CIFAR-10 (4 images par classe)



Le dataset CIFAR-10

60 000 images au total : 50 000 pour l'entraînement et 10 000 pour test

10 classes équilibrées : 5 000 images par classe d'entraînement, 1000 par classe de test



Préparation des données

Normalisation : pixels ramenés de [0,255] à [0,1]

```
# --- 2.1 Normalisation des images (float32, [0, 1]) ---  
X_train = X_train_raw.astype('float32') / 255.0  
X_test  = X_test_raw.astype('float32') / 255.0
```

Partition : 80% train / 20% validation / test fourni séparément

```
# --- 2.2 Partition train / validation (80/20) ---  
VAL_SPLIT = 0.2  
n_val = int(len(X_train) * VAL_SPLIT)  
  
X_val  = X_train[-n_val:]  
y_val  = y_train_raw[-n_val:]  
X_train = X_train[:-n_val]  
y_train = y_train_raw[:-n_val]
```

Encodage one-hot des labels (ex: chat → [0,0,0,1,0,0,0,0,0])

```
# --- 2.3 Encodage one-hot des labels ---  
y_train_ohe = keras.utils.to_categorical(y_train, NUM_CLASSES)  
y_val_ohe   = keras.utils.to_categorical(y_val,   NUM_CLASSES)  
y_test_ohe  = keras.utils.to_categorical(y_test_raw, NUM_CLASSES)
```

Architecture du modèle 1

Schéma de l'architecture : 3 blocs Conv2D + MaxPooling + Dense

Rôle de chaque couche :

- Conv2D → détecte des features (contours, formes)
- MaxPooling → réduit la taille, garde l'essentiel
- Dense → classifie à partir des features extraites

30 époques, batch size 64, optimiseur Adam



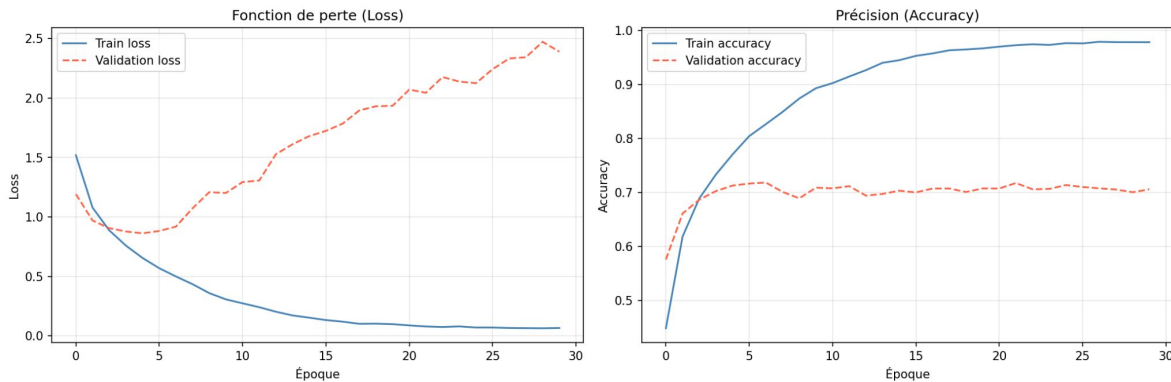
Resultat Modèle 1 + Overfitting

*La précision sur l'ensemble d'entraînement (train accuracy) monte à **97.8%** mais celle sur l'ensemble de validation (validation accuracy) stagne à **70.6%***

La perte de validation (val_loss) remonte dès l'époque 5 alors que la perte d'entraînement (train_loss) continue de baisser → overfitting classique

Conclusion : le modèle **mémorise** les données d'entraînement sans généraliser

Modèle 1 — CNN de base (30 époques)



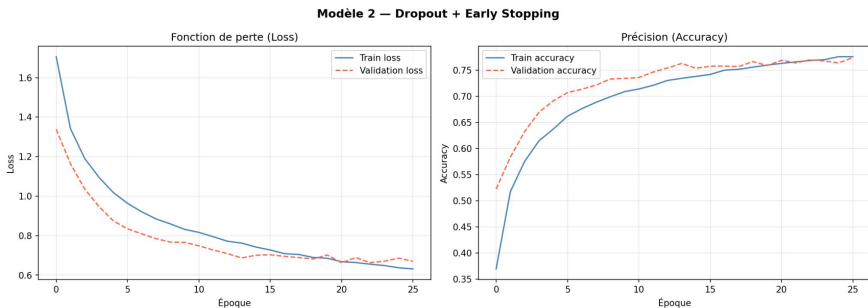
Modèle 2 : Dropout + Early Stopping

Explication Dropout : A chaque étape, on désactive aléatoirement X% des neurones → force le réseau à ne pas dépendre d'un seul chemin

Explication Early Stopping : On arrête si val_loss ne s'améliore plus pendant 5 époques

Résultat : train ~77.6%, val ~77.4%, test **76.5%** en 25 époques

L'écart train/val est quasi nul → l'overfitting est résolu, mais accuracy modeste



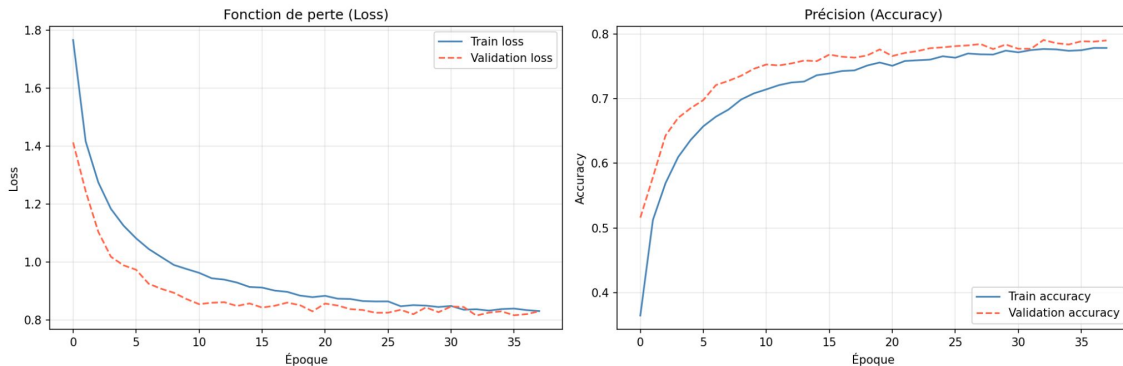
Modèle 3 : Dropout + L2 + Early Stopping

Explication L2 : On pénalise les poids trop grands dans la loss → évite que le modèle sur-spécialise

Résultat : train ~77.8%, val ~79.0%, test **78.3%** en 38 époques

Légère amélioration par rapport au modèle 2. La val_accuracy dépasse même légèrement la train_accuracy, signe que la régularisation fonctionne bien

Modèle 3 — Dropout + L2 + Early Stopping



Modèle 4 : Data Augmentation + Architecture profonde

Data Augmentation : on génère des variantes des images (rotations, flips, zoom) → le modèle voit artificiellement plus de diversité

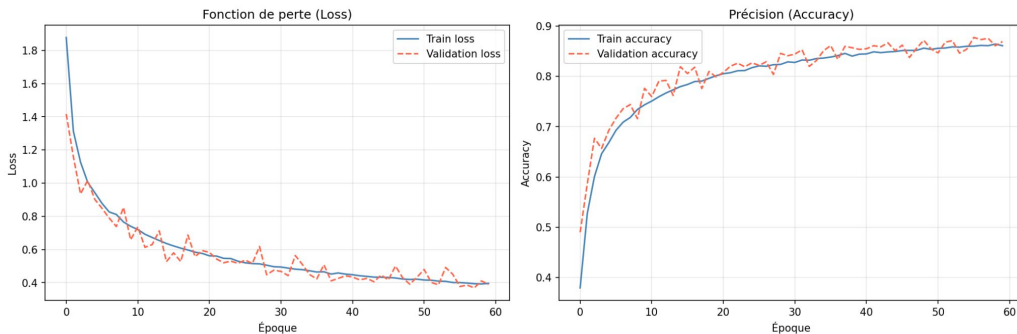
Architecture plus profonde : 2 Conv2D par bloc au lieu d'une + BatchNormalization

BatchNorm : normalise les activations entre les couches → stabilise et accélère l'apprentissage

Résultat : train ~86.1%, val ~86.9%, test **86.7%** en 60 époques

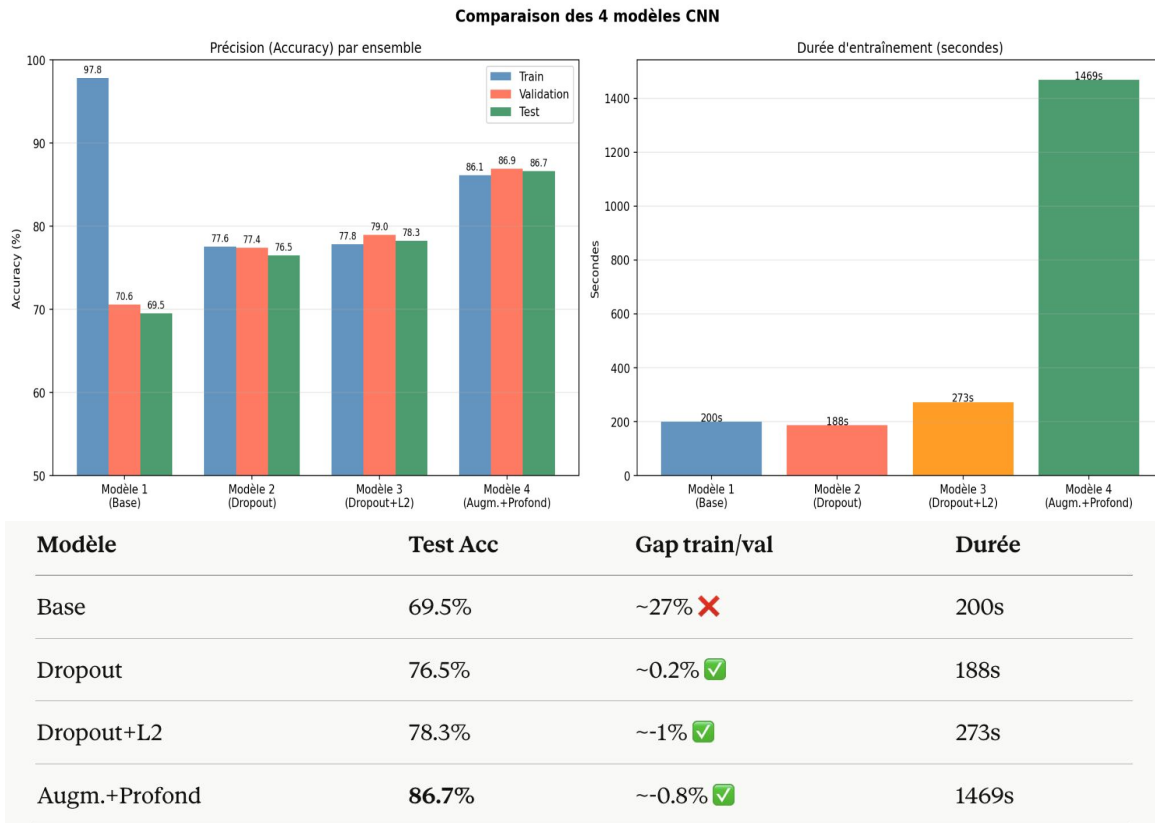
Meilleur modèle de loin, mais au prix d'un temps d'entraînement 7x plus long (1469s vs ~200s)"

Modèle 4 — Data Augmentation + Architecture profonde



Comparaison des 4 modèles

Chaque technique apporte une amélioration. La data augmentation combinée à une architecture plus riche donne le meilleur résultat.

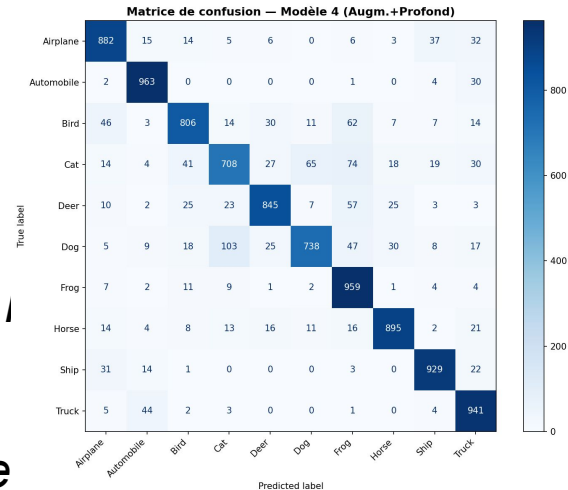


Analyse détaillée du meilleur modèle

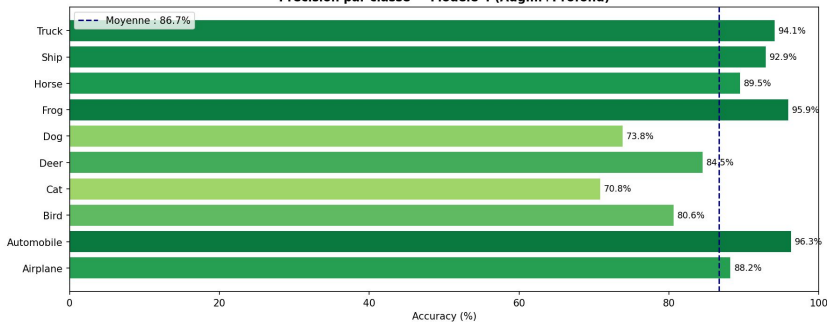
Les classes les mieux reconnues sont Automobile (96.3%), Ship (92.9%) et Frog (95.9%)

Les plus difficiles sont Cat (70.8%) et Dog (73.8%) → souvent confondues

Un oiseau prédit comme automobile, un cerf prédit comme chat → compréhensible vu la basse résolution



Précision par classe — Modèle 4 (Augm.+Profond)



Prédictions du Modèle 4 (Augm.+Profond)



Limites et pistes d'amélioration

Limites :

- Images 32×32 très basse résolution, information limitée
- Modèle 4 très long à entraîner (~25 min)
- Hyperparamètres non optimisés systématiquement
- Cat et Dog restent difficiles même pour le meilleur modèle

Pistes :

- **Transfer Learning** (ResNet, EfficientNet pré-entraîné sur ImageNet) → atteindre >95%
 - **Learning rate scheduler** (réduire le LR automatiquement)
 - **Augmentation plus agressive** (Cutout, Mixup)
- 

Conclusion

Résumé en 3 points :

1. Le CNN de base overfite massivement → techniques de régularisation indispensables
2. Dropout + Early Stopping suffisent à éliminer l'overfitting
3. Data Augmentation + architecture profonde = meilleur compromis performance/généralisation

Notre meilleur modèle atteint 86.7% de précision sur 10 classes sans aucun modèle pré-entraîné, ce qui est un bon résultat pour une architecture CNN entraînée from scratch

